Contents

SRE Practices — Deep Dive Notes **1**
1. What is SRE? . . . 1
2. Service Level Terminology . . . 1
   - SLI — Service Level Indicator . . . 1
   - SLO — Service Level Objective . . . 2
   - SLA — Service Level Agreement . . . 2
   - Error Budget . . . 2
3. Toil — Identification and Elimination . . . 2
   - What is Toil? . . . 2
   - Toil Examples . . . 3
   - Toil Reduction Strategy . . . 3
4. Incident Management . . . 3
   - Incident Severity Classification . . . 3
   - Incident Response Roles . . . 3
   - Incident Communication Template . . . 4
5. Blameless Post-Mortems . . . 4
   - Why Blameless? . . . 4
   - Post-Mortem Structure . . . 4
   - 5 Whys Example . . . 4
6. Chaos Engineering . . . 5
   - Principles of Chaos (Netflix Chaos Monkey origin) . . . 5
   - Tools . . . 5
   - Resilience Patterns . . . 5
7. Capacity Planning . . . 6
   - Capacity Planning Process . . . 6
8. On-Call Best Practices . . . 6
   - Healthy On-Call Culture . . . 6
   - Runbooks . . . 6

 Resources . . . 7

SRE Practices — Deep Dive Notes

1. What is SRE?

Site Reliability Engineering applies software engineering principles to infrastructure and operations. Coined at Google (2003). Key book: *Site Reliability Engineering* (free at sre.google).

Core philosophy: - Toil (manual repetitive work) should be < 50% of SRE time - Error budgets balance velocity with reliability - Everything that can be automated should be automated - Post-mortems are blameless — systems, not people, are at fault

SRE vs DevOps: - DevOps: a culture and set of practices - SRE: an implementation of DevOps principles with specific tools (SLOs, error budgets, on-call) - “SRE is what you get when you hire a software engineer to do operations” — Google

---

2. Service Level Terminology

SLI — Service Level Indicator

A *quantitative measure* of some aspect of service behavior.

Good SLIs directly measure user experience:

Service Type	Example SLI
Request-based	% of requests completing successfully in < 500ms

Service Type	Example SLI
Data pipeline	% of records processed without error within SLA window
Storage	Proportion of reads returning correct data
Batch job	% of jobs completing within scheduled window

Bad SLIs: CPU %, memory usage (these are causes, not symptoms of user pain)

SLO — Service Level Objective

The *target value* for an SLI.

```
SLI:  ratio of successful requests (status < 500, latency < 500ms)
SLO:  99.5% over a rolling 28-day window

SLO should reflect user happiness threshold:
  - Below 99% → users notice and are angry
  - 99.5% → acceptable with occasional complaints
  - 99.99% → imperceptible to most users
```

Setting the right SLO: 1. Start with what users actually experience (measure first) 2. Talk to product/business about acceptable impact 3. Start slightly below measured baseline 4. Tighten only after you can reliably hit the target 5. Avoid over-engineering: 99.999% is almost never necessary

SLA — Service Level Agreement

A *contractual commitment* to the customer, usually with financial penalties for breach. SLA should be more lenient than SLO (internal buffer).

```
Internal:  SLO = 99.9%  (what you aim for)
External:  SLA = 99.5%  (what you promise customers)
Buffer:    0.4%         (wiggle room for investigations)
```

Error Budget

```
Error Budget = 1 - SLO (as percentage of time)

99.9% SLO:
  Error budget = 0.1% = 43.8 minutes/month downtime

99.5% SLO:
  Error budget = 0.5% = 3.65 hours/month

99.99% SLO:
  Error budget = 0.01% = 4.38 minutes/month
```

Error budget policy: - Budget > 50% remaining → deploy freely, run experiments, accept risk - Budget 25-50% → slow down risky deployments, investigate reliability issues - Budget < 25% → freeze non-critical deployments, focus on reliability - Budget exhausted → full freeze until next window; SRE team gets priority

---

3. Toil — Identification and Elimination

What is Toil?

Manual, repetitive, automatable, tactical work that scales linearly with service growth. Not the same as overhead (meetings, reviews) or project work.

Toil characteristics: - Manual (human required) - Repetitive (done over and over) - Automatable (machine could do it) - Reactive (triggered by external event) - No lasting value (same work tomorrow) - Scales with service size

Toil Examples

```
High Toil:
  - Manually restarting pods when out-of-memory
  - Manual certificate renewals
  - Copying configuration values between environments
  - Manually approving every deployment
  - Manually answering "what's the status of X?" questions
  - Resizing disks when they fill up

Low/No Toil (projects):
  - Building auto-remediation for OOM restarts
  - Implementing cert-manager for automated renewals
  - Building a self-service deployment platform
  - Writing runbooks that prevent manual interventions
```

Toil Reduction Strategy

```
1. Measure: Track time spent on each repetitive task
2. Prioritize: Highest frequency × time per instance = biggest ROI
3. Automate first attempt: Script → cron → event-driven
4. Eliminate root cause: Why does the disk keep filling up?
5. Track reduction: Monthly toil hours metric
```

---

4. Incident Management

Incident Severity Classification

```
SEV-1: Complete outage, all users affected, revenue impact
  Example: Payment service down, all transactions failing
  Response: Immediate, full team, external communication

SEV-2: Major feature broken, significant user impact
  Example: Login failures for 40% of users
  Response: < 15 minutes, dedicated IC + responders

SEV-3: Degraded performance, some users affected
  Example: API latency 3x normal, no errors
  Response: < 1 hour, 1-2 engineers

SEV-4: Minor issue, minimal user impact, workaround exists
  Example: Non-critical report generation slow
  Response: Business hours, create ticket
```

Incident Response Roles

```
IC (Incident Commander):
  - Coordinates response, not necessarily the one fixing it
  - Manages communications, time-boxes investigation steps
  - Makes escalation decisions

Technical Lead:
  - Drives technical investigation and mitigation
  - Reports status to IC every 15 min

Communications Lead:
  - Updates status page, internal stakeholders
  - Drafts customer communications
  - Manages exec escalation
```

```
Scribe:
  - Documents timeline, decisions, commands run
  - Creates post-mortem skeleton during incident
```

Incident Communication Template

```
IMPACT:   [What users are experiencing]
STATUS:   [Investigating / Identified / Monitoring / Resolved]
SCOPE:    [% users affected, which regions/features]
TIMELINE: HH:MM - [what happened / what we did]
NEXT:     [what we're doing in next 30 minutes]
ETA:      [estimated resolution, if known]
```

---

5. Blameless Post-Mortems

Why Blameless?

When people fear blame, they hide information, minimize their involvement, and don't tell the full story. This prevents learning. Systems fail, not people.

"If we fire the person who caused the outage, we fire the person who knows most about that failure mode." — Google SRE

Post-Mortem Structure

```
1. Incident Summary
   - What happened, impact, duration, severity

2. Timeline (chronological)
   - Detection, key actions, resolution
   - What helped, what slowed us down

3. Root Cause Analysis
   - Use 5 Whys technique
  - Go 5 levels deep: Why did service fail? → Why was DB slow? → Why was query slow? → Why was there no i

4. Contributing Factors (not blame)
   - "The runbook didn't mention this failure mode"
   - "Alert fired 10 minutes after user reports began"
   - "No staging equivalent existed for this component"

5. Action Items (specific, assigned, dated)
   - [Owner] [Due Date] Add index on orders.user_id
   - [Owner] [Due Date] Create alert for query > 5s
   - [Owner] [Due Date] Update runbook with new failure mode

6. Lessons Learned
   - What worked well
   - What we'd do differently
```

5 Whys Example

```
Problem: Payment service was down for 22 minutes

Why 1: Payment service was unreachable
  → Database connection pool exhausted

Why 2: Connection pool exhausted
```

```
  → Slow queries holding connections

Why 3: Slow queries due to full table scan
  → Missing index on transactions.created_at column

Why 4: Index was missing
  → New query added without index in last deployment

Why 5: New query added without index review
  → No automated EXPLAIN ANALYZE in CI for query changes

Root cause: Lack of query performance gate in CI pipeline
Action: Add automated slow query detection to deployment pipeline
```

---

6. Chaos Engineering

Principles of Chaos (Netflix Chaos Monkey origin)

```
1. Define "steady state" (what does healthy look like)
2. Hypothesize that steady state continues in experimental group
3. Introduce variables: kill pods, inject latency, drain nodes
4. Disprove hypothesis (find weaknesses)
5. Fix weaknesses, repeat
```

Tools

```
LitmusChaos:    Kubernetes-native, ChaosHub library of experiments
  - Pod kill, pod CPU stress, pod memory hog
  - Node drain, network partition, disk fill

Chaos Monkey:   Netflix original, kills random EC2 instances
Gremlin:        SaaS platform, safe chaos engineering
k6:             Load testing + chaos combination

Chaos GameDay:
  - Scheduled controlled chaos experiment
  - Team assembled, monitoring ready
  - Progressively escalate: 1 pod → all pods in AZ → full region
  - Document findings, improve resilience
```

Resilience Patterns

```
Circuit Breaker:
  CLOSED (normal) → too many failures → OPEN (reject all requests) →
  HALF-OPEN (test one request) → success → CLOSED

Retry with Exponential Backoff:
  Retry after 1s, 2s, 4s, 8s, 16s (max)
  Always add jitter to prevent thundering herd

Bulkhead:
  Isolate failures: separate thread pools or connection pools per dependency
  One slow external API doesn't exhaust all connections

Timeout:
  Always set timeouts on every external call
  Default: 1-5s for synchronous calls; longer for async/batch

Fallback:
```

```
  Return cached result, default value, or degraded response when dependency fails
  Example: recommendation service down → return top-selling items
```

---

7. Capacity Planning

Capacity Planning Process

```
1. Measure current utilization
   - CPU, memory, disk, network per service
   - Peak vs average load profiles

2. Forecast growth
   - Historical traffic trends (YoY, QoQ)
   - Business projections (new customer growth, product launches)
   - Seasonal patterns (Black Friday, end-of-quarter)

3. Identify bottlenecks
   - Which resource will hit capacity first?
   - What is the scaling path? (vertical vs horizontal)

4. Plan headroom
   - Run at ~60-70% average capacity (30-40% headroom for spikes)
   - 2x current capacity for HA and rollouts without degradation

5. Automate scaling
   - HPA for pods (CPU/memory/custom metrics)
   - Cluster Autoscaler for nodes
   - KEDA for event-driven scaling

6. Load test
   - Simulate projected peak traffic
   - Find failure modes before users do
   - Validate autoscaling behavior
```

---

8. On-Call Best Practices

Healthy On-Call Culture

```
Rotation: Maximum 2 weeks per quarter per person (Google standard)
Compensation: On-call pay for nights/weekends
Response time: SEV-1 = 5 min, SEV-2 = 15 min
Handoff: Synchronous handoff with active incidents
Toil target: On-call should spend < 25% time on incidents

Alert quality metrics:
  - What % of pages required human action? (target: 100%)
  - What % were self-resolved? (target: 0%)
  - Mean time between pages? (target: > 2 hours)
```

Runbooks

Every alert must have a runbook. A good runbook:

```
Title: [Alert Name]
Severity: [Critical/Warning]
Impact: [What users experience]

Symptoms:
```

```
  - [What the alert fires on]
  - [Other things you might observe]

Likely Causes:
  1. [Most common cause] → [check this command]
  2. [Second cause] → [check this command]

Diagnosis Steps:
  1. Check dashboard: [link]
  2. Run: kubectl top pods -n production
  3. Check recent deployments: [link]

Remediation:
  - [Fix for cause 1]
  - [Fix for cause 2]
  - Escalate to: [team] if unresolved after 30 min

Post-Incident: File post-mortem if SEV-1/2
```

Resources

- https://sre.google/sre-book/table-of-contents/ — Google SRE Book (FREE)
- https://sre.google/workbook/table-of-contents/ — Google SRE Workbook (FREE)
- https://litmuschaos.io/ — LitmusChaos docs
- https://principlesofchaos.org/ — Chaos Engineering principles
- https://www.pagerduty.com/resources/ — PagerDuty incident response guides